Exploring the React Project

Continuing where we left off, we're poised to create a React project. Creating a React project is made simple with the command line tool provided by React. This tool rapidly generates starter files, enabling you to bootstrap your project and create a React application with minimal effort. It essentially takes the fuss out of initiating a React app.

Prerequisites

First, ensure that Node.js is installed on your PC, as it's a requirement to create a React application.

Creating a React Application

To initiate a new React application, execute the command:

```
npx create-react-app my-app
```

Here, `create-react-app` is the command to generate the project, and `my-app` is your chosen project name, customizable as per your preference.

Running the React Application

Once your application is created, navigate into the directory of your new application using:

```
cd my-app
```

Then, run the following command to start the application:

```
npm start
```

Quick Recap

1. **Create a new React application**: Use `npx create-react-app my-app`
2. **Navigate into your React application**: `cd my-app`
3. **Start your application**: `npm start`

Benefits of the `create-react-app` Command

The command `npx create-react-app my-app` creates a brand-new React application without requiring you to deal with build configurations, which is quite convenient. The command builds, compiles, and transpiles the application, starts the server, watches for source code changes, and rebuilds using hot reload. No manual work needed!

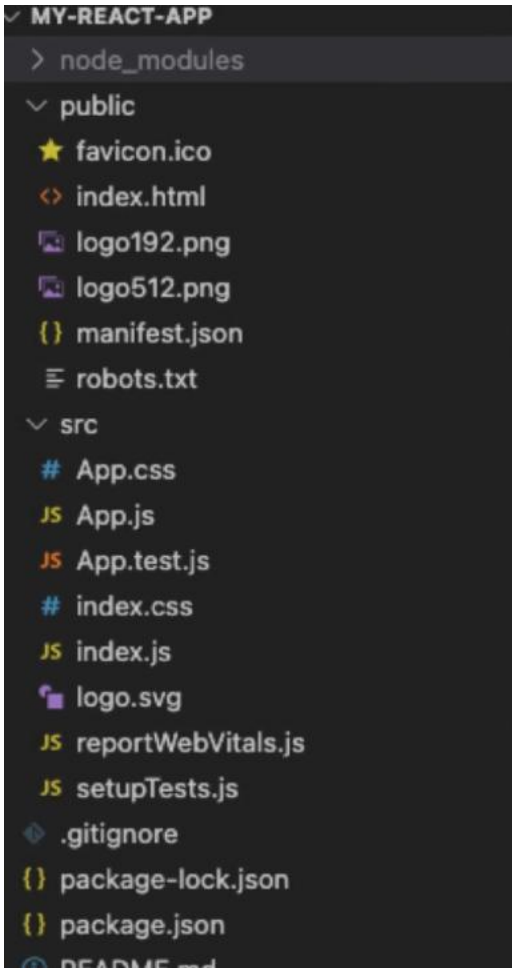
Customizing the Port

By default, React uses port 3000. To change this, you can use:

```
set PORT=5100 && npm start
```

This ensures the server listens on Port 5100, accessed at `http://localhost:5100`.

Exploring Project Files



Let's dive into the anatomy of your React project files. First on the list is the `node_modules` directory. This crucial folder houses all the libraries and packages installed via the Node Package Manager (npm). Whenever you initialize a new React application, an automatic `npm install` command is run, populating this directory with all the necessary dependencies to make your React app functional.

Throughout our lessons, we'll also explore how to enrich your project by installing additional external libraries. The `npm install` command will facilitate this, adding the new modules into the `node_modules` directory so they're readily available for use within your application.

Next, let's look at the `public` directory. This folder contains your `favicon.ico`, which serves as the browser tab icon for your application, typically the React logo by default. The `index.html` file here provides the basic HTML structure and is where the Document Object Model (DOM) for your React application gets initialized.

You'll also notice some logo files in various sizes; you're free to remove these if they're not needed. In addition, the `manifest.json` file provides metadata about your project, including its name, author, and icon. Then we have the `robots.txt` file, which instructs search engine bots whether they should crawl your application or not.

Moving on to the `src` directory: this is the heart of your application. Here, you'll find `App.css`, the global stylesheet for your React app, and `App.js`, the main JavaScript component that orchestrates your application's logic. There's also an `App.test.js` file, which is where you'll place unit tests for your application. `index.css` contains styling specific to the `index.html` file, while `index.js` serves as the root component that attaches itself to `index.html` to start rendering React components.

You may also see files like `logo.svg`, reports, and web vitals, among others. These are often auxiliary and may be skipped for the scope of this course.

Don't overlook the `.gitignore` file. If you plan to use Git or any other version control systems, this file specifies which files and folders should not be tracked. We also have `package.json` and `package-lock.json` files. The former defines your project's dependencies, metadata, and scripts, while the latter locks down the version numbers of your installed packages to ensure consistent installations across different setups.

Lastly, a `README.md` file is automatically created when you initialize your React project, providing a convenient place for you to document your project's objectives, setup instructions, and other useful information.

In summary, while most of your development work will reside in the `src` directory, it's beneficial to understand the roles and purposes of all the other files and directories that make up a React project.

With this overview complete, we're all set to jump into some actual coding!